

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №11.

Тема: «Поиск данных с помощью хэш-таблиц»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 23

Выполнила: студентка группы РИС-25-26

Лаптева Елена Владимировна

Принял: доц. Полякова О.А.

Пермь, 2026

Задание

Постановка задачи

1. Создать динамический массив из записей (в соответствии с вариантом), содержащий не менее 100 элементов. Для заполнения элементов массива использовать ДСЧ.
2. Предусмотреть сохранение массива в файл и загрузку массива из файла.
3. Предусмотреть возможность добавления и удаления элементов из массива (файла).
4. Выполнить поиск элемента в массиве по ключу в соответствии с вариантом. Для поиска использовать хэш-таблицу.
5. Подсчитать количество коллизий при размере хэш-таблицы 40, 75 и 90 элементов.

Анализ задачи

ШАГ 0: Запуск программы

ШАГ 1: Пользователь видит меню и выбирает действие (цифра 0-8)

ШАГ 2: В зависимости от выбора:

1. Если 1 (генерация):
 - Очищаем массив
 - Запускаем ДСЧ с сидом 42
 - В цикле 100+ раз создаём Person со случайными данными
 - Добавляем каждую запись в динамический массив
 - Печатаем "Сгенерировано N записей"
2. Если 2 (сохранение):
 - Открываем файл persons.dat для записи
 - Пишем количество записей в первой строке
 - Для каждой записи пишем: имя|дата|телефон
 - Закрываем файл
3. Если 3 (загрузка):
 - Открываем файл для чтения
 - Читаем количество записей
 - Очищаем текущий массив
 - В цикле читаем строки, разбиваем по "|"
 - Создаём Person и добавляем в массив
4. Если 4 (добавить):
 - Спрашиваем у пользователя имя, дату, телефон
 - Создаём Person
 - Если в массиве нет места → увеличиваем его в 2 раза
 - Добавляем запись в конец

5. Если 5 (удалить):
 - Спрашиваем телефон
 - Ищем в массиве запись с таким телефоном
 - Если нашли → сдвигаем все последующие элементы на 1 влево
 - Уменьшаем счётчик записей
 6. Если 6 (поиск через хэш):
 - Если массив пуст → выходим
 - Создаём хэш-таблицу размера 101
 - Вставляем ВСЕ записи из массива в хэш-таблицу
 - Спрашиваем телефон для поиска
 - Вычисляем хэш телефона → индекс в таблице
 - Ищем в списке по этому индексу
 - Если нашли → выводим данные, иначе → "Не найден"
 7. Если 7 (анализ коллизий):
 - Если массив пуст → выходим
 - Для каждого размера из [40, 75, 90]:
 - Создаём новую хэш-таблицу
 - Вставляем все записи из массива
 - При каждой вставке: если ячейка уже не пуста → коллизия++
 - Считаем $\alpha = \text{количество_записей} / \text{размер_таблицы}$
 - Выводим строку: размер | коллизии | α
 8. Если 8 (вывести всё):
 - Если массив пуст → пишем "Массив пуст"
 - Иначе печатаем заголовок таблицы
 - В цикле выводим каждую запись с форматированием
 9. Если 0 → выходим из цикла, программа завершается
- ШАГ 3: Возврат к меню (пока пользователь не выбрал 0)

Код

```
#include <iostream>
#include <string>
#include <fstream>
#include <random>
#include <sstream>
#include <iomanip>
```

```
using namespace std;
```

// СТРУКТУРА ЧЕЛОВЕК

```
struct Person {  
    string name;      // Поле: имя человека (например, "Ivan5")  
    string birthDate; // Поле: дата рождения в формате  
    "ДД.ММ.ГГГГ"  
    string phone;     // Поле: номер телефона (уникальный ключ для  
    поиска)  
  
    // Перегрузка оператора == для сравнения двух записей  
    // Два человека считаются равными, если у них совпадает  
    телефон  
    bool operator==(const Person& other) const {  
        return phone == other.phone; // Сравниваем только телефоны  
    }  
};
```

// ОДНОСВЯЗНЫЙ СПИСОК

```
template <typename T>  
class List {  
    // Узел списка  
    struct Node {  
        T data;  
        Node* next;  
  
        // Конструктор узла: инициализируем данные и обнуляем  
        указатель  
        explicit Node(const T& val) : data(val), next(nullptr) {}  
    };  
  
    Node* head; // Указатель на первый узел списка  
    size_t sz;  // Счётчик: сколько элементов сейчас в списке  
  
public:  
    // Конструктор по умолчанию: создаём пустой список  
    List() : head(nullptr), sz(0) {}  
  
    // Деструктор: при удалении списка освобождаем всю память  
    ~List() { clear(); }
```

```
// Запрещаем копирование списка
List(const List&) = delete; // Запрет копирования через
конструктор
List& operator=(const List&) = delete; // Запрет копирования через
присваивание
```

```
// Добавить элемент в конец списка
void push_back(const T& val) {
    Node* newNode = new Node(val); // Создаём новый узел

    if (!head) { // Если список пуст (head == nullptr)
        head = newNode; // Новый узел становится головой
    }
    else { // Если список не пуст
        Node* curr = head; // Начинаем обход с головы
        while (curr->next) // Идём по списку, пока не найдём
последний узел
            curr = curr->next; // Переходим к следующему узлу
        curr->next = newNode; // Прицепляем новый узел к
последнему
    }
    sz++; // Увеличиваем счётчик элементов
}
```

```
// Проверка: пуст ли список?
bool empty() const {
    return head == nullptr; // Пуст, если голова указывает на nullptr
}
```

```
// Получить текущее количество элементов в списке
size_t size() const {
    return sz; // Возвращаем сохранённый счётчик
}
```

// ИТЕРАТОРЫ

```
struct Iterator {
    Node* ptr; // Указатель на текущий узел при обходе
```

```
// Разыменованное итератора: получить данные текущего узла
```

```

T& operator*() { return ptr->data; }

// Префиксный инкремент: перейти к следующему узлу
Iterator& operator++() {
    ptr = ptr->next; // Двигаем указатель вперёд
    return *this;    // Возвращаем себя для цепочки вызовов
}

// Сравнение итераторов: нужны для условия окончания цикла
for
bool operator!=(const Iterator& other) const {
    return ptr != other.ptr; // Не равны, если указывают на разные
узлы
}
};

// Получить итератор на начало списка (первый элемент)
Iterator begin() { return { head }; } // Возвращаем итератор,
указывающий на head

// Получить итератор на конец списка (за последним элементом)
Iterator end() { return { nullptr }; } // Конец обозначается nullptr

// Удалить первый элемент, равный val (использует operator==
типа T)
bool remove(const T& val) {
    Node** curr = &head; // Двойной указатель: чтобы менять head,
если удаляем первый элемент

    while (*curr) {
        if ((*curr)->data == val) { // Если нашли нужный элемент
            Node* toDelete = *curr; // Запоминаем узел для
удаления
            *curr = toDelete->next; // "Вырезаем" узел из цепочки
delete toDelete; // Освобождаем память
            sz--; // Уменьшаем счётчик
            return true;
        }
    }
}

```

```

        curr = &((*curr)->next); // Переходим к указателю на
следующий узел
    }
    return false; // Элемент не найден
}

```

```

// Очистить список: удалить все узлы и освободить память
void clear() {
    while (head) {
        Node* tmp = head;
        head = head->next;
        delete tmp;
    }
    sz = 0; // Сбрасываем счётчик
}
};

```

// ДИНАМИЧЕСКИЙ МАССИВ

```

class DynamicArray {
    Person* data;
    size_t count;           // Сколько записей реально добавлено
(логический размер)
    size_t capacity;       // Сколько записей влезает без перевыделения
(физический размер)

```

```

    // Приватный метод: увеличить массив до newCap элементов
    void resize(size_t newCap) {
        Person* newData = new Person[newCap]; // Выделяем НОВЫЙ
блок памяти большего размера

```

```

        // Копируем все старые элементы в новый блок
        for (size_t i = 0; i < count; ++i)
            newData[i] = data[i]; // Поэлементное копирование
(вызывается оператор присваивания)

```

```

        delete[] data; // Освобождаем СТАРЫЙ блок памяти (чтобы не
было утечки!)
        data = newData; // Перенаправляем указатель на новый блок
        capacity = newCap; // Обновляем значение ёмкости

```

```
}
```

```
public:
```

```
    // Конструктор: создаём массив с начальной ёмкостью (по умолчанию 128)
```

```
    explicit DynamicArray(size_t initCap = 128) : count(0), capacity(initCap) {
```

```
        data = new Person[capacity]; // Выделяем память под capacity элементов типа Person
```

```
    }
```

```
    // Деструктор: освобождаем память при удалении объекта
```

```
    ~DynamicArray() {
```

```
        delete[] data;
```

```
    }
```

```
    // Запрещаем копирование массива (аналогично списку: избегаем двойного free)
```

```
    DynamicArray(const DynamicArray&) = delete;
```

```
    DynamicArray& operator=(const DynamicArray&) = delete;
```

```
    // Добавить новую запись в конец массива
```

```
    void add(const Person& p) {
```

```
        if (count >= capacity) // Если места в текущем блоке нет
```

```
            resize(capacity * 2); // Увеличиваем ёмкость в 2 раза
```

```
            data[count++] = p; // Копируем запись в свободную ячейку и увеличиваем счётчик
```

```
    }
```

```
    // Удалить запись по номеру телефона (ключ)
```

```
    bool removeByPhone(const string& phone) {
```

```
        // Линейный поиск: перебираем все элементы
```

```
        for (size_t i = 0; i < count; ++i) {
```

```
            if (data[i].phone == phone) { // Нашли нужную запись
```

```
                // Сдвигаем все последующие элементы на 1 позицию
```

```
влево
```

```
                for (size_t j = i; j < count - 1; ++j)
```

```
                    data[j] = data[j + 1]; // Копирование соседнего элемента
```

```
                count--; // Уменьшаем логический размер массива
```



```

        return true;
    }
}
return false; // Запись с таким телефоном не найдена
}

// Геттеры и операторы доступа
size_t size() const { return count; } // Получить количество записей
const Person& operator[](size_t i) const { return data[i]; } // Доступ
по индексу (только чтение)
void clear() { count = 0; } // Сбросить счётчик (не освобождает
память, просто "забывает" данные)

// Сохранить массив в текстовый файл
void saveToFile(const string& filename) const {
    ofstream out(filename); // Создаём поток для записи в файл
    if (!out) { // Проверка: успешно ли открылся файл?
        cerr << "Ошибка открытия файла для записи!\n"; // Вывод
ошибки в stderr
        return; // Прерываем выполнение метода
    }
    out << count << "\n"; // Первая строка: количество записей

    // Записываем каждую запись в формате: имя|дата|телефон
    for (size_t i = 0; i < count; ++i)
        out << data[i].name << "|" << data[i].birthDate << "|" <<
data[i].phone << "\n";

    out.close(); // Закрываем файл
}

// Загрузить массив из текстового файла
void loadFromFile(const string& filename) {
    ifstream in(filename); // Создаём поток для чтения из файла
    if (!in) { // Проверка: существует ли файл?
        cerr << "Файл не найден!\n";
        return;
    }
}

```

```
clear(); // Очищаем текущий массив перед загрузкой новых
данных
```

```
size_t n; // Переменная для хранения количества записей в
файле
```

```
if (!(in >> n)) return; // Читаем число; если не удалось -
выходим
```

```
in.ignore(); // Пропускаем символ перевода строки после числа
```

```
string line; // Буфер для чтения каждой строки файла
```

```
for (size_t i = 0; i < n; ++i) { // Цикл по количеству записей
```

```
getline(in, line); // Читаем всю строку до '\n'
```

```
stringstream ss(line); // Создаём поток для парсинга строки
```

```
Person p; // Создаём временную запись
```

```
// Разбиваем строку по разделителю '|'
```

```
getline(ss, p.name, '|'); // Часть до первого '|' → имя
```

```
getline(ss, p.birthDate, '|'); // Часть между '|' → дата
```

```
getline(ss, p.phone, '|'); // Часть после второго '|' → телефон
```

```
add(p); // Добавляем распарсенную запись в массив (с
возможным resize)
```

```
}
```

```
in.close(); // Закрываем файл
```

```
}
```

```
// Вывести все записи на экран в виде таблицы
```

```
void print() const {
```

```
if (count == 0) { // Проверка на пустой массив
```

```
cout << "Массив пуст.\n";
```

```
return;
```

```
}
```

```
// Печатаем заголовок таблицы с форматированием
```

```
cout << left << setw(20) << "Name" << setw(12) << "BirthDate"
<< "Phone\n";
```

```
cout << string(45, '-') << "\n"; // Разделительная линия из 45
дефисов
```

```
// Выводим каждую запись
```

```
for (size_t i = 0; i < count; ++i)
```

```

        cout << left << setw(20) << data[i].name    // Имя, выровнено
влево, ширина 20
        << setw(12) << data[i].birthDate            // Дата, ширина 12
        << data[i].phone << "\n";                  // Телефон, без
ограничения ширины
    }
};

```

```

// ХЭШ-ТАБЛИЦА (МЕТОД ЦЕПОЧЕК, ИСПОЛЬЗУЕТ List<T>)
class HashTable {
    int m;                // Размер таблицы (количество корзин/ячеек)
    List<Person>* buckets; // Массив списков: каждая ячейка — это
список для коллизий
    int collisions;       // Счётчик: сколько раз произошла коллизия
при вставке

```

```

    // Приватная хэш-функция: преобразует строку (телефон) в число
    size_t hashFunc(const string& key) const {
        size_t h = 0; // Начальное значение хэша
        // Полиномиальный хэш: каждый символ умножается на
степень 31
        for (char c : key) // Перебираем каждый символ строки
            h = h * 31 + static_cast<size_t>(c); // h = h*31 + код_символа
        return h; // Возвращаем полученное хэш-значение (беззнаковое,
чтобы избежать отрицательных индексов)
    }

```

```

public:
    // Конструктор: создаём массив из m пустых списков
    explicit HashTable(int size) : m(size), collisions(0) {
        buckets = new List<Person>[size]; // Динамический массив
списков
    }

```

```

    // Деструктор
    ~HashTable() {
        for (int i = 0; i < m; ++i) // Для каждой ячейки таблицы
            buckets[i].clear();    // Очищаем список (удаляем все узлы)
        delete[] buckets; // Освобождаем сам массив списков
    }

```

```

}

// Запрещаем копирование хэш-таблицы (избегаем двойного free)
HashTable(const HashTable&) = delete;
HashTable& operator=(const HashTable&) = delete;

// Вставить запись в хэш-таблицу
void insert(const Person& p) {
    // Шаг 1: вычисляем хэш телефона и берём остаток от деления
    // на размер таблицы
    size_t idx = hashFunc(p.phone) % static_cast<size_t>(m);

    // Шаг 2: если ячейка уже не пуста → это коллизия,
    // увеличиваем счётчик
    if (!buckets[idx].empty())
        collisions++;

    // Шаг 3: добавляем запись в список соответствующей ячейки
    // (в конец)
    buckets[idx].push_back(p);
}

// Поиск записи по телефону
Person* search(const string& phone) {
    // Вычисляем индекс ячейки по хэшу телефона
    size_t idx = hashFunc(phone) % static_cast<size_t>(m);

    // Перебираем все элементы в списке этой ячейки (используем
    // наш итератор)
    for (auto& p : buckets[idx])
        if (p.phone == phone) // Сравниваем телефоны (используется
            // operator==)
            return &p; // Нашли, возвращаем указатель на запись

    return nullptr; // Не нашли, возвращаем нулевой указатель
}

// Удалить запись по телефону
bool remove(const string& phone) {

```

```

// Вычисляем индекс ячейки
size_t idx = hashFunc(phone) % static_cast<size_t>(m);

    // Создаём временный объект с нужным телефоном (для
сравнения через operator==)
    Person temp;
    temp.phone = phone;

    // Вызываем remove у списка нужной ячейки
    return buckets[idx].remove(temp); // Возвращаем true, если
удаление прошло успешно
}

// Геттер для получения количества коллизий
int getCollisions() const {
    return collisions;
}
};

// ГЕНЕРАЦИЯ ДАННЫХ С ПОМОЩЬЮ ДАТЧИКА
СЛУЧАЙНЫХ ЧИСЕЛ
void generateData(DynamicArray& arr, int n) {
    arr.clear(); // Очищаем массив перед генерацией новых данных

    // Инициализируем генератор псевдослучайных чисел с
фиксированным сидом
    mt19937 rng(42); // Сид 42 гарантирует, что при каждом запуске
будут ОДИНАКОВЫЕ "случайные" данные

    // Создаём распределения для разных полей
    uniform_int_distribution<int> distDay(1, 28);    // День: от 1 до 28
    uniform_int_distribution<int> distMonth(1, 12);  // Месяц: от 1 до
12
    uniform_int_distribution<int> distYear(1985, 2005); // Год: от 1985
до 2005
    uniform_int_distribution<int> distDigit(0, 9);   // Цифра: от 0 до 9

    // Список имён для выбора

```

```
const vector<string> names = { "Ivan", "Maria", "Petr", "Anna",  
"Alex", "Elena",  
                                "Dmitry", "Olga", "Sergey", "Natalia" };  
uniform_int_distribution<int> distName(0, names.size() - 1); //  
Распределение для выбора имени из списка
```

```
// Генерируем n записей  
for (int i = 0; i < n; ++i) {  
    Person p; // Создаём пустую запись  
  
    // Формируем имя: случайное имя из списка + номер для  
уникальности  
    p.name = names[distName(rng)] + to_string(i);  
  
    // Формируем дату в формате ДД.ММ.ГГГГ  
    p.birthDate = to_string(distDay(rng)) + "." + // День  
        (distMonth(rng) < 10 ? "0" : "") + to_string(distMonth(rng)) + "."  
+ // Месяц с ведущим нулём если нужно  
        to_string(distYear(rng)); // Год  
  
    // Формируем телефон: "+7" + 10 случайных цифр  
    p.phone = "+7";  
    for (int j = 0; j < 10; ++j) // Добавляем 10 цифр  
        p.phone += to_string(distDigit(rng));  
  
    arr.add(p); // Добавляем сгенерированную запись в  
динамический массив  
}  
cout << "Сгенерировано " << n << " записей.\n";  
}
```

```
// АНАЛИЗ КОЛЛИЗИЙ ДЛЯ РАЗМЕРОВ ТАБЛИЦЫ 40, 75, 90  
void testCollisions(const DynamicArray& arr) {  
    int sizes[] = { 40, 75, 90 }; // Три размера таблицы для сравнения  
по заданию  
  
    // Выводим заголовок таблицы результатов  
    cout << "\nАнализ коллизий\n";
```

```

    cout << left << setw(10) << "Size" << setw(15) << "Collisions" <<
    setw(15) << "Load Factor\n";
    cout << string(40, '-') << "\n";

    // Перебираем каждый размер таблицы
    for (int m : sizes) {
        HashTable ht(m); // Создаём новую хэш-таблицу текущего
        размера

        // Вставляем ВСЕ записи из массива в эту таблицу
        for (size_t i = 0; i < arr.size(); ++i)
            ht.insert(arr[i]); // При вставке автоматически считается
        количество коллизий

        // Считаем коэффициент загрузки:  $\alpha = N / M$ 
        double load = static_cast<double>(arr.size()) / m;

        // Выводим строку результатов: размер | коллизии |
        коэффициент загрузки
        cout << left << setw(10) << m
            << setw(15) << ht.getCollisions()
            << setw(15) << fixed << setprecision(2) << load << "\n";
    }
}

int main() {
    setlocale(LC_ALL, "ru");

    DynamicArray db(128); // Создаём динамический массив с
    начальной ёмкостью 128
    const string FILENAME = "persons.dat"; // Имя файла для
    сохранения/загрузки

    int choice; // Переменная для хранения выбора пользователя в
    меню
    do {
        // Выводим меню с доступными опциями
        cout << "\n1. Сгенерировать массив (>=100)\n"
            << "2. Сохранить в файл\n"

```

```

    << "3. Загрузить из файла\n"
    << "4. Добавить запись\n"
    << "5. Удалить запись по телефону\n"
    << "6. Поиск по телефону (хэш-таблица)\n"
    << "7. Анализ коллизий (40, 75, 90)\n"
    << "8. Вывести весь массив\n"
    << "0. Выход\n"
    << "Выбор: ";
    cin >> choice; // Считываем выбор пользователя

// Обрабатываем выбор через switch
switch (choice) {
case 1: { // Генерация данных
    int n;
    cout << "Количество записей (мин. 100): ";
    cin >> n;
    if (n < 100) n = 100; // Гарантируем минимум 100 записей по
условию
    generateData(db, n); // Вызываем функцию генерации
    break;
}
case 2: // Сохранение в файл
    db.saveToFile(FILENAME);
    cout << "Сохранено.\n";
    break;

case 3: // Загрузка из файла
    db.loadFromFile(FILENAME);
    cout << "Загружено " << db.size() << " записей.\n";
    break;

case 4: { // Ручное добавление записи
    Person p; // Создаём пустую запись
    cout << "Имя: "; cin >> p.name; // Считываем имя
    cout << "Дата (ДД.ММ.ГГГГ): "; cin >> p.birthDate; //
Считываем дату
    cout << "Телефон: "; cin >> p.phone; // Считываем телефон
    db.add(p); // Добавляем в массив
    cout << "Добавлено.\n";
}
}

```



```

        break;
    }
    case 5: { // Удаление по телефону
        string phone;
        cout << "Телефон для удаления: ";
        cin >> phone;
        if (db.removeByPhone(phone)) // Пытаемся удалить
            cout << "Удалено.\n";
        else
            cout << "Не найдено.\n"; // Если телефон не найден
        break;
    }
    case 6: { // Поиск через хэш-таблицу
        if (db.size() == 0) { // Проверка: пуст ли массив?
            cout << "Массив пуст!\n";
            break;
        }
        HashTable ht(101); // Создаём хэш-таблицу оптимального
размера (простое число)
        for (size_t i = 0; i < db.size(); ++i) // Заполняем таблицу всеми
записями
            ht.insert(db[i]);
        string phone;
        cout << "Телефон для поиска: ";
        cin >> phone;
        Person* res = ht.search(phone); // Ищем запись
        if (res) // Если найдена
            cout << "Найден: " << res->name << ", " << res->birthDate
<< ", " << res->phone << "\n";
        else // Если не найдена
            cout << "Не найден.\n";
        break;
    }
    case 7: // Анализ коллизий
        if (db.size() == 0) { // Проверка на пустой массив
            cout << "Сначала сгенерируйте данные!\n";
            break;
        }
        testCollisions(db); // Запускаем анализ для 40/75/90

```

```
        break;

    case 8: // Вывод всех записей
        db.print();
        break;

    case 0: // Выход из программы
        cout << "Выход.\n";
        break;

    default: // Обработка неверного ввода
        cout << "Неверный выбор.\n";
    }
} while (choice != 0); // Повторяем меню, пока пользователь не
выберет 0

return 0;
}
```

Результат выполнения программы

```
C:\Users\Elena Lapteva\source\repos\labs\64\Debug\labs.exe
1. Сгенерировать массив (>=100)
2. Сохранить в файл
3. Загрузить из файла
4. Добавить запись
5. Удалить запись по телефону
6. Поиск по телефону (хэш-таблица)
7. Анализ коллизий (40, 75, 90)
8. Вывести весь массив
0. Выход
Выбор: 1
Количество записей (мин. 100): 100
Сгенерировано 100 записей.

1. Сгенерировать массив (>=100)
2. Сохранить в файл
3. Загрузить из файла
4. Добавить запись
5. Удалить запись по телефону
6. Поиск по телефону (хэш-таблица)
7. Анализ коллизий (40, 75, 90)
8. Вывести весь массив
0. Выход
Выбор: 2
Сохранено.

1. Сгенерировать массив (>=100)
2. Сохранить в файл
3. Загрузить из файла
4. Добавить запись
5. Удалить запись по телефону
6. Поиск по телефону (хэш-таблица)
7. Анализ коллизий (40, 75, 90)
8. Вывести весь массив
0. Выход
Выбор: 3
Загружено 100 записей.

1. Сгенерировать массив (>=100)
2. Сохранить в файл
3. Загрузить из файла
4. Добавить запись
5. Удалить запись по телефону
6. Поиск по телефону (хэш-таблица)
7. Анализ коллизий (40, 75, 90)
```

```
persons.dat - Блокнот
Файл  Правка  Формат  Вид  Справка
100
Anna0|21.012.2001|+77551410048
Anna1|19.02.1997|+70097892019
Maria2|15.04.1997|+70402563102
Natalia3|3.03.1992|+77613595408
Dmitry4|2.03.1999|+70999583300
Petr5|20.03.1999|+71648019321
Dmitry6|15.04.2000|+72551098749
Anna7|21.012.2003|+79305150938|
Anna8|24.04.2000|+75392652128
Maria9|12.1.1986|+77312008177
Olga10|3.010.2001|+79361988643
Ivan11|19.05.1986|+73675628543
Maria12|22.9.2005|+77527240574
Maria13|6.06.1985|+70864355691
Petr14|22.05.1997|+72290526189
Dmitry15|3.04.2001|+78482148853
Sergey16|9.011.1987|+79122640838
Anna17|15.02.1985|+74426123291
Anna18|20.07.1989|+74309292264
Olga19|28.02.1991|+70269540023
Natalia20|5.03.1998|+75449525607
Anna21|23.03.1989|+73469635807
Sergey22|6.04.1988|+76085362075
Sergey23|3.05.1989|+71963359816
Anna24|26.02.2000|+75862261895
Elena25|28.04.1996|+70985986132
Anna26|26.9.1999|+78847868091
Olga27|17.09.2003|+70319690213
Elena28|19.09.1995|+79217024317
Anna29|21.9.1998|+76355053622
Petr30|12.12.1997|+74896473565
Dmitry31|7.011.1995|+77425076019
Natalia32|26.012.1994|+78370993459
Elena33|3.12.2005|+72331823413
Elena34|20.12.1993|+79500266931
Petr35|10.1.1995|+77861773522
```

```
Выбор: 4
Имя: Helen
Дата (ДД.ММ.ГГГГ): 27.01.2006
Телефон: +72912345678
Добавлено.

1. Сгенерировать массив (>=100)
2. Сохранить в файл
3. Загрузить из файла
4. Добавить запись
5. Удалить запись по телефону
6. Поиск по телефону (хэш-таблица)
7. Анализ коллизий (40, 75, 90)
8. Вывести весь массив
0. Выход
Выбор: 6
Телефон для поиска: +72912345678
Найден: Helen, 27.01.2006, +72912345678

1. Сгенерировать массив (>=100)
2. Сохранить в файл
3. Загрузить из файла
4. Добавить запись
5. Удалить запись по телефону
6. Поиск по телефону (хэш-таблица)
7. Анализ коллизий (40, 75, 90)
8. Вывести весь массив
0. Выход
Выбор: 5
Телефон для удаления: +72912345678
Удалено.
```

Выбор: 7

Анализ коллизий

Size	Collisions	Load Factor
------	------------	-------------

40	61	2.50
75	41	1.33
90	42	1.11

1. Сгенерировать массив (≥ 100)
2. Сохранить в файл
3. Загрузить из файла
4. Добавить запись
5. Удалить запись по телефону
6. Поиск по телефону (хэш-таблица)
7. Анализ коллизий (40, 75, 90)
8. Вывести весь массив
9. Выход

Выбор: 8

Name	BirthDate	Phone
------	-----------	-------

Anna0	21.012.2001	+77551410048
Anna1	19.02.1997	+70097892019
Maria2	15.04.1997	+70402563102
Natalia3	3.03.1992	+77613595408
Dmitry4	2.03.1999	+70999583300
Petr5	20.03.1999	+71648019321
Dmitry6	15.04.2000	+72551098749
Anna7	21.012.2003	+79305150938
Anna8	24.04.2000	+75392652128
Maria9	12.1.1986	+77312008177
Olga10	3.010.2001	+79361988643
Ivan11	19.05.1986	+73675628543
Maria12	22.9.2005	+77527240574
Maria13	6.06.1985	+70864355691
Petr14	22.05.1997	+72290526189
Dmitry15	3.04.2001	+78482148853
Sergey16	9.011.1987	+79122640838
Anna17	15.02.1985	+74426123291
Anna18	20.07.1989	+74309292264
Olga19	28.02.1991	+70269540023
Natalia20	5.03.1998	+75449525607
Anna21	23.03.1989	+73469635807
Sergey22	6.04.1988	+76085362075
Sergey23	3.05.1989	+71963359816

Natalia71	21.8.1996	+79783224212
Natalia72	7.08.1990	+78676032106
Alex73	24.6.1995	+75388595087
Alex74	1.02.1997	+74776678291
Dmitry75	18.012.1985	+75635409735
Elena76	12.10.1994	+72658697149
Sergey77	2.010.1995	+74096433265
Petr78	4.01.1992	+74151416611
Dmitry79	25.5.1992	+74569161208
Maria80	5.04.1992	+70051543283
Maria81	10.04.1995	+70074660581
Natalia82	8.01.1992	+78807519253
Natalia83	22.03.1995	+73547700121
Olga84	15.11.1999	+77501843532
Maria85	6.09.1990	+70538203512
Sergey86	20.08.1992	+73764405575
Elena87	16.08.2000	+71323386652
Anna88	17.012.1995	+79201213241
Sergey89	23.02.1988	+71088005046
Natalia90	12.2.2005	+75998089241
Olga91	21.10.1999	+75554287219
Anna92	15.06.1999	+77251767205
Elena93	9.012.1988	+75503341505
Natalia94	23.04.1989	+70216071544
Dmitry95	3.010.1993	+79887941979
Natalia96	2.03.1989	+79775087187
Olga97	24.010.1989	+71182665933
Sergey98	23.05.1997	+74473846357
Alex99	10.07.1995	+78335509265

1. Сгенерировать массив (>=100)
 2. Сохранить в файл
 3. Загрузить из файла
 4. Добавить запись
 5. Удалить запись по телефону
 6. Поиск по телефону (хэш-таблица)
 7. Анализ коллизий (40, 75, 90)
 8. Вывести весь массив
 0. Выход
- Выбор: 0
Выход.